

Real Python pandas Cheat Sheet

Practice at realpython.com with hands-on coding exercises, quizzes, and a personal AI mentor.

Loading Data

- Import pandas as `pd`; everyone does
- A `DataFrame` is a table of `Series` columns that share one index
- `read_csv()` also takes URLs and zipped files
- pandas 3 needs Python 3.11 or newer

Install pandas

```
$ python -m pip install pandas matplotlib
```

Read a CSV File

```
import pandas as pd

df = pd.read_csv("sales.csv")
df = pd.read_csv(
    "sales.csv",
    usecols=["date", "region", "units"],
    parse_dates=["date"],
)
```

Build a DataFrame by Hand

```
df = pd.DataFrame({
    "city": ["Oslo", "Lima"],
    "temp": [4, 19],
})
```

Read Other Formats

```
pd.read_excel("sales.xlsx")  # Needs openpyxl
pd.read_json("sales.json")
pd.read_parquet("sales.parquet")
```

Save Your Results

```
df.to_csv("clean.csv", index=False)
```

Learn More on realpython.com/search:

`pandas read_csv` · `pandas dataframe`

Inspecting Data

- A non-null count below the row count means missing values
- Integer columns with gaps load as `float64`

Take a First Look

```
df.head(3)      # First 3 rows
df.tail()       # Last 5 rows
df.sample(2)    # 2 random rows
df.shape        # (8, 5)
df.columns      # Column labels
```

Check Types and Stats

```
df.info()       # Dtypes + non-null counts
df.dtypes       # Units → float64
df.describe()   # Count, mean, std, ...
```

Count Categories

```
>>> df["region"].value_counts()
region
North    3
South    3
East     2
Name: count, dtype: int64
```

Learn More on realpython.com/search:

`explore dataset` · `pandas dtypes`

Selecting Data

- `.loc` uses labels, and its slices include the end
- `.iloc` uses positions, and its slices exclude the end
- One label gives a `Series`, a list gives a `DataFrame`

Pick Columns

```
df["units"]              # Series
df[["region", "units"]]  # DataFrame
```

Pick Rows and Cells

```
df.loc[0, "region"]    # 'North'
df.loc[0:2, "units"]   # Rows 0, 1, and 2
df.iloc[0, 1]          # 'North'
df.iloc[:2, -1]        # First 2 prices
df.iloc[-1]            # Last row
```

Learn More on realpython.com/search:

`pandas loc iloc` · `pandas indexing`

Filtering and Sorting

- Combine conditions with `&`, `|`, and `~`, and wrap each one in parentheses
- Use `@name` to reach Python variables in `.query()`

Filter With Boolean Masks

```
df[df["units"] > 10]           # 3 rows
df[(df["region"] == "North")
   & (df["units"] > 5)]        # 2 rows
df[df["product"].isin(["Widget", "Gizmo"])]
df[df["product"].str.startswith("G")]
```

Filter With a Query String

```
min_units = 8
df.query("units >= @min_units")
df.query("region == 'East' and units > 10")
```

Sort Rows

```
df.sort_values("units", ascending=False)
df.nlargest(3, "units")      # Top 3 rows
```

Learn More on realpython.com/search:

`pandas filter` · `pandas sort`

Cleaning Data

- Most methods return a new object, so assign the result back
- Change cells with `.loc`; pandas 3 ignores chained assignment like `df["a"][0] = 1`
- `pd.to_numeric(s, errors="coerce")` turns bad values into `NaN`

Find Missing Values

```
>>> df.isna().sum()
date       0
region     0
product    0
units      1
price      1
dtype: int64
```

Drop or Fill Missing Values

```
df.dropna()                  # 6 rows left
df.dropna(subset=["units"])  # 7 rows left
df["units"] = df["units"].fillna(0)
df["price"] = df["price"].fillna(
    df["price"].median()
)
```

Fix Data Types

```
df["date"] = pd.to_datetime(df["date"])
df["units"] = df["units"].astype(int)
df["region"] = (
    df["region"].astype("category")
)
```

Tidy Up and Add Columns

```
df = df.drop_duplicates()
df["region"] = df["region"].str.strip()
df = df.rename(columns={"qty": "units"})
df["revenue"] = df["units"] * df["price"]
df.loc[0, "units"] = 13  # Set one cell
```

Unify Inconsistent Labels

```
df["region"] = df["region"].replace(
    {"N": "North", "S": "South"}
)
```

Learn More on realpython.com/search:

`data cleaning` · `pandas dropna` · `missing data`

Grouping and Aggregating

- `.groupby()` splits rows, applies a function, and combines the results
- Add `.reset_index()` to get a flat table back
- Month-end resampling uses `"ME"` since pandas 2.2

Aggregate One Column

```
>>> df.groupby("region")["units"].sum()
region
East     23
North    24
South    24
Name: units, dtype: int64
```

Compute Several Stats

```
df.groupby("region").agg(
    total=("revenue", "sum"),
    orders=("units", "count"),
)
```

Build a Pivot Table

```
df.pivot_table(
    index="region", columns="product",
    values="units", aggfunc="sum",
)
```

Group by Time

```
df.resample("D", on="date")["units"].sum()
```

Learn More on realpython.com/search:

`pandas groupby` · `pivot table`

Plotting

- `.plot()` draws with Matplotlib, so install it too
- Notebooks show plots inline; scripts need `plt.show()`

From Raw CSV to Chart

```
import matplotlib.pyplot as plt

# Which product earns the most?
(pd.read_csv("sales.csv")
   .dropna(subset=["units", "price"])
   .assign(revenue=lambda d:
           d["units"] * d["price"])
   .groupby("product")["revenue"]
   .sum()
   .sort_values()
   .plot(kind="barh", title="Revenue"))
plt.tight_layout()
plt.savefig("revenue.png")
```

Pick a Plot Kind

```
df.plot(x="date", y="units")  # Line
df["units"].plot(kind="hist")
df.plot.scatter(x="price", y="units")
df["product"].value_counts().plot.pie()
```

Learn More on realpython.com/search:

`pandas plot` · `matplotlib`

Become a Python and AI expert with your free personal learning plan at realpython.com/learning-plan

Real Python